

[← Go Back](#)

Hardware

Nicla Voice



Tutorials

Door Intruder Detector
Using ML with the Nicla
Voice

Audio Analysis with
Machine Learning and the
Nicla Voice

**Glass-Breaking Detection
Using Edge Impulse &
Nicla Voice**

Motion Detection with Nicla
Voice and Machine
Learning Tools

Nicla Voice User Manual

Home / Hardware / Nicla Voice / **Glass-Breaking Detection Using Edge Impulse & Nicla Voice**

Glass-Breaking Detection Using Edge Impulse & Nicla Voice

This application note describes creating a glass-breaking detection system using the Arduino Nicla Voice, Machine Learning Tools, and the Edge Impulse platform.

Author · Taddy Ho · Chung & Christopher Méndez · Last revision · 12/18/2024

ON THIS PAGE

Introduction

Goals

Hardware and Software Requirements +

Edge Impulse and Nicla Voice Versatility

Glass Break Machine Learning Model Development +

Deployment +

Glass Breaking Detection

Expanding Glass Breaking Detector +

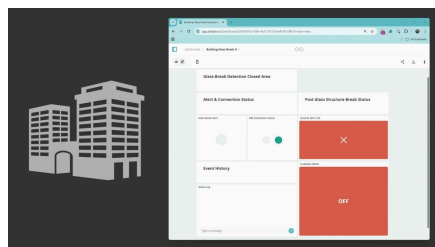
Full Glass-Breaking Detector Resources

Conclusion

Next Steps

Introduction

The Nicla Voice and Edge Impulse offer powerful tools for monitoring environments like spaces with glass structures. This system allows timely detection and responses for environments like buildings, vehicles, or specialized facilities. You can create scalable AI solutions for security needs by leveraging these tools.



This application note will show you how to build and deploy a machine learning model on Nicla Voice with Edge Impulse that detects glass-

[Help](#)

and used for training for efficient sound classification.

Goals

This application note will show you how to create a glass-breaking detection system using the Nicla Voice and Edge Impulse. The objectives are:

- Process audio data of glass-breaking sounds and background noise using the **DCASE** dataset.

- Train a machine learning model to distinguish glass-breaking sounds from other noises reliably.

- Deploy the trained model to the Nicla Voice for on-device inference, removing the need for external computation.

- Showcase the model's real-time detection capabilities through practical examples.

At the end of the application note, you will have all resources and details for implementing a glass-breaking detection system in various environments.

Hardware and Software Requirements

Hardware Requirements

- Nicla Voice**

- USB-C® cable** (x1)

- 3.7 V LiPo battery (Optional)

Portenta H7

USB-C® cable (x1)

Wi-Fi® Access Point (x1)

Software Requirements

Arduino Machine Learning Tools

Edge Impulse® CLI, to easily flash any Machine Learning model created with the Arduino Machine Learning Tools, we can install the Edge Impulse® Cli tool following these [instructions](#).

Nicla Voice audio **firmware** to upload recordings to Machine Learning Tools.

Arduino IDE 1.8.10+, **Arduino IDE 2**, or **Arduino Cloud Editor** (Optional).

The **Arduino Create Agent**

The **Arduino Cloud**. If you do not have an account, you can create one for free inside cloud.arduino.cc.

Additional Resources

Complete example code and Edge Impulse project:

[Glass-breaking detection model](#)

[Base glass-breaking detector example](#)

[Extended glass-breaking detector with Arduino Cloud](#)

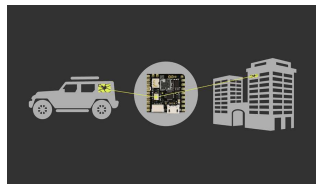
Dataset resources:

Pre-recorded glass-breaking sound samples (or access to the **DCASE** dataset).

A speaker audio playback:

Edge Impulse and Nicla Voice Versatility

Edge Impulse and Nicla Voice provide a highly flexible and adaptable solution for various monitoring applications, including glass structure monitoring systems.



Nicla Voice & Edge Impulse
integration versatility

These applications, such as monitoring buildings, vehicles, and public spaces, are some examples that can develop into a comprehensive system integrated with the Arduino Cloud:

Buildings: Identify glass-breaking incidents in residential, commercial, or industrial properties. Programmed alerts can trigger lockdown protocols, notify security teams, and log incidents in real time.

Vehicles: Detect windshield breakage and provide real-time SOS signals, vehicle location, or license plate details. This feature is particularly beneficial for improving vehicle security.

Custom Environments:
Deploy in specific setups like public transportation, specialized labs, or sensitive

related incidents require immediate attention.

This adaptability provides comprehensive coverage and flexible response mechanisms for various scenarios. Additionally, integrating BLE communication and Arduino Cloud makes the system more versatile and responsive.

If you are new to the Nicla Voice, it is recommended that you review the [Nicla Voice User Manual](#) to familiarize yourself with its features and setup before continuing with this application note.

Glass Break Machine Learning Model Development

The application will be based on a preconfigured example model created by Aurelien Lequertier that is available on the [Arduino Machine Learning Tools](#).

This example detects anomalous sounds of glass breaking using the [DCASE rare sound events dataset](#). With this example, you will learn how to build and deploy a glass-breaking detection system.

Data Collection

Data collection is the foundation of any machine learning project. This application note will use an existing [DCASE dataset \(Detection and Classification of Acoustic Scenes](#)

challenges of recording actual glass-breaking sounds.

This dataset provides a broad collection of audio samples designed for rare sound event detection, including glass-breaking sounds and background noise. It includes:

Mixtures: 1500 of 30 second audio mixtures, totaling 12 hours and 30 minutes of audio, created by combining isolated events with background recordings.

Isolated Events: 474 unique events across three target categories: baby crying, **glass breaking**, and gunshot.

Background Recordings: 1121 of 30 second recordings of ambient sounds totaling 9 hours and 20 minutes of audio.

For this application note, we focus on glass-breaking sounds. From this dataset, *313 audio samples* are used for training and *54 samples* for testing. These samples represent two main categories:

Glass-breaking events

Non-glass-breaking ambient noise

Using the **DCASE** dataset helps you to have a diverse range of audio conditions, improving the robustness of the final model.

The audio data was preprocessed with **Edge Impulse**. Its user interface allows you to manage audio files, including cropping unnecessary segments and splitting longer recordings into manageable pieces.

This preprocessing step ensures that only relevant data is provided into the training pipeline, maximizing the

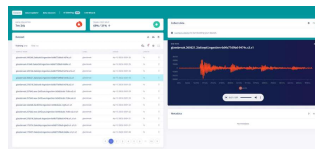
preprocessing step includes following steps:

Audio Segmentation: Splitting long audio recordings into smaller, manageable clips for more effective training.

Noise Handling: Removing background noise or irrelevant segments to ensure the dataset focuses on the target sounds.

Normalization: Ensuring consistent audio levels across all samples to improve model reliability. Using Edge Impulse GUI, you can crop unnecessary segments, annotate audio clips, and simplify this process.

Using Edge Impulse, you can simplify these preprocessing steps with tools to crop unnecessary segments, annotate audio clips, and visualize the processed data. The dataset can be found within the **Data acquisition** section in the Edge Impulse platform.



Dataset Overview in Edge Impulse

The above image shows the dataset overview in Edge Impulse, where you can see the collected audio samples for glass-breaking events. The data is divided into training and testing sets to ensure the model's generalization.

Additionally, Edge Impulse allows you to collect data directly from your devices, either through a USB connection or wirelessly. This capability is particularly useful for iterative model improvement, allowing you to add new audio

ensures that the model remains accurate and effective throughout its lifecycle.

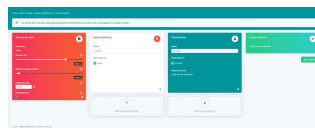
Data Processing

In this step, you will use an audio processing block specifically optimized for the NDP120 processor integrated into the Nicla Voice. This block uses a *Fast Fourier Transform (FFT)* to convert audio signals from the time domain into the frequency domain, extracting meaningful features for classification.

Dimensionality reduction techniques help visualize and cluster features, effectively distinguishing between glass-breaking and non-glass-breaking sounds.

The first image below shows the time series configuration and audio processing block in Edge Impulse. Configurable parameters, such as window size and frequency, are important in extracting the most meaningful features for audio data classification.

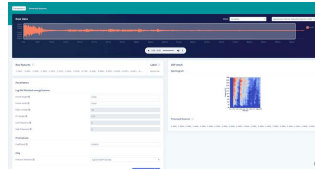
These configurations are available in the **Create Impulse** section of the Edge Impulse platform.



Signal Processing
Configuration

The processing block includes customizable settings such as window size and feature scaling, allowing users to fine-tune signal

flexibility provides optimal feature extraction and improved classification accuracy.



Feature Generation and Visualization

The second image shows the output of feature generation. The data visualization reveals clusters corresponding to glass-breaking and non-glass-breaking audio samples.

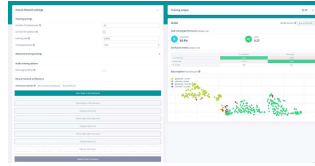
This helps us understand how well the system can separate these two types of sounds, which can be accessed under the **Parameters** tab of the **Syntiant** section in Edge Impulse.

Model Training

You will use a dense neural network architecture that is well-suited for audio classification tasks. The configuration ensures compatibility with the hardware, using a 968 millisecond audio window for processing requirements.

Edge Impulse's Eon tuner can help automatically optimize model parameters, simplifying the process for you without extensive machine learning expertise.

The model achieved an accuracy of 89 % during training, showing its capability to differentiate between glass-breaking and background noises. This can be found within the



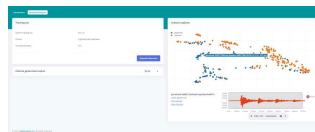
Model Testing Results

Architecture: Dense neural network

Window Size: 968 milliseconds

Optimization: The **Eon tuner** automatically optimized signal processing and model parameters.

Accuracy: The model achieved an accuracy of 89% on the training set, demonstrating robust performance.

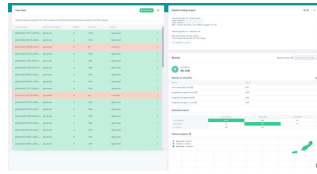


Model Training Settings

The image shows the training settings and the neural network architecture used in Edge Impulse. This window can be found within the **Generate features** tab from **Syntiant** section in the Edge Impulse platform. The Eon tuner can help you optimize hyperparameters like learning rate, number of neurons, and layer configurations to achieve the best balance between model size, accuracy, and latency.

Model Testing

After training, the model is evaluated using a test dataset that includes previously unseen audio samples. This evaluation helps ensure the model generalizes adequately to new



Model Testing Results

The image above shows the testing results with an accuracy of **99.12 %**. This can be found within the **Model testing** section in the Edge Impulse platform.

The confusion matrix highlights the model's capability to classify audio samples correctly while identifying potential areas for improvement:

Correctly classified "glassbreak" events: 94.4 %

Correctly classified "unknown" samples: 100 %

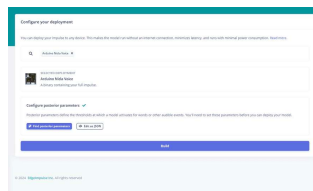
Misclassifications could have happened for short-duration glass-breaking sounds, suggesting the need for additional data or refinement of the processing pipeline to improve robustness. Regular updates to the dataset with new samples can address such limitations and improve performance.

 The model performance can be affected if the application is implemented in a very different environment than the one used for training. It is recommended that the datasets be provided with new samples and the model be retrained for a new and upgraded deployment.

Deployment

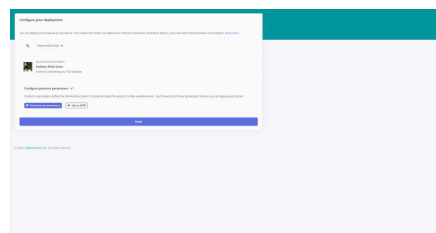
After training the model, it can be deployed to the Nicla Voice using **binary firmware builder from Edge Impulse**. This builder creates firmware that includes the trained model and the necessary scripts for on-device inference, allowing operation without internet connectivity.

To begin, navigate to the **Deployment** section in the Edge Impulse platform. The following image shows the deployment configuration screen, where you can select Nicla Voice as the deployment target.



Configure Deployment for
Nicla Voice

Once the Nicla Voice is selected as the deployment target, click on **Build** to start the firmware build process. The animation below provides an overview of how this process looks:



After the build is complete, the firmware files will automatically download. You can retrieve them later from the **Latest build** tab in the **Deployment** section if needed.



Nicla Voice Firmware Build Files

The downloaded firmware comes as a compressed file. Extract it to your preferred directory. You will find several files in the extracted folder, including a directory named `ndp120`.

This folder contains the NDP120 processor firmware and the trained machine learning model. There are four scripts for each supported operating system:

flash_(os): Runs the MCU firmware and model flash scripts.

flash_(os)_mcu: Flashes only the MCU firmware.

flash_(os)_model: Flashes the NDP firmware, DSP firmware (if not present), and updates the model.

format_(os)_ext_flash: Erases the external flash memory.

`(os)` refers to the operating system: linux, mac, or windows.

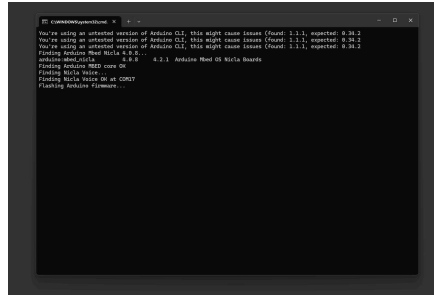
To install the required dependencies, run the `install_lib` script for your operating system. This script only needs to be run once. The model update process uses the Python package `pyserial`.

After preparing the firmware, connect the Nicla Voice to your computer.

Use the corresponding script to automatically proceed with the firmware update and model upload. For Windows users, for example, the following command can be used in a terminal:

1 flash_windows

The animation below shows the firmware update process and uploading the trained model:



If you prefer to update manually, navigate to the `ndp120` directory and run the following commands in sequence:

```
1 ./syntiant-uploader-win se
```

```
1 ./syntiant-uploader-win se
```

```
1 ./syntiant-uploader-win se
```



If you are not familiar with updating the NDP120 processor firmware and uploading the trained machine learning model, please refer to [this section](#) of the Nicla Voice User Manual.

For a new model deployment with

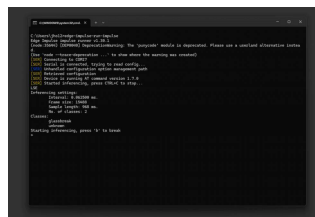
`ei_model.synpkg`, which you can find inside the downloaded folder with yours.

Running with Edge Impulse CLI

Once the Nicla Voice is updated with the firmware and model, you can run the trained glass-breaking detector model using the Edge Impulse CLI:

```
1 edge-impulse-run-impulse
```

If everything is set up correctly, the terminal will display information similar to the image below:



Build & Flash for Nicla Voice

Running with Arduino IDE

Alternatively, you can monitor the inference process using an Arduino script uploaded to the Nicla Voice.

Open the Arduino IDE and use the following example script to observe results in the Serial Monitor:

```
1 #include "NDP.h"
2
3 const bool lowestPower =
4
5 void ledBlueOn(char* label) {
6     nicla::leds.begin();
7     nicla::leds.setColor(kBlue);
8     delay(200);
9     nicla::leds.setColor(kOff);
10    if (!lowestPower) {
11        Serial.println(label);
12    }
13    nicla::leds.end();
14 }
15
16 void ledGreenOn() {
17     nicla::leds.begin();
18     nicla::leds.setColor(kGreen);
19     delay(200);
20     nicla::leds.setColor(kOff);
21     nicla::leds.end();
22 }
23
24 void ledRedBlink() {
25     while (1) {
26         nicla::leds.begin();
27         nicla::leds.setColor(kRed);
28         delay(200);
```

Glass Breaking Detection

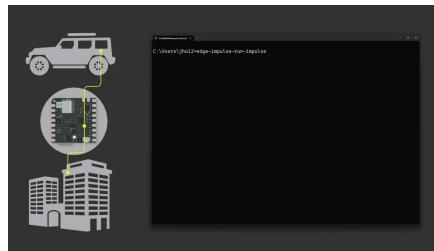
With the firmware and model successfully deployed, you can simulate glass-breaking events by playing recorded sounds through a speaker or testing with real-world glass.

The trained model processes the sound in real-time and detects glass-breaking events, showing its capability.

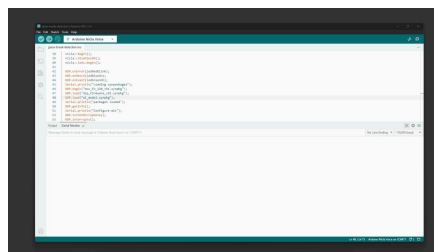
Although the sound of the real-world glass structure breaking presents the best method to test and validate the trained model, please take precautions and safety measures to avoid possible accidents or injuries.

The Edge Impulse CLI provides real-time monitoring and classification results. The Arduino IDE Serial Monitor also allows you to observe the same information.

Below is an example of a real-time glass break classification result:



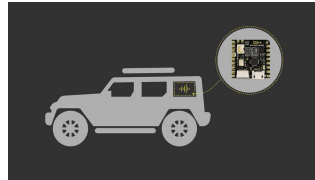
You can also use the Arduino IDE's Serial Monitor to observe similar results found within the Edge Impulse CLI.



With the machine learning model trained, validated, and deployed onto the Nicla Voice, the system is ready for real-world use case applications. As an example, the following section shows how to validate the setup and test the system in practical scenarios.

Expanding Glass

Nicla Voice is now capable of detecting glass-breaking sounds using a trained machine learning model. You can create an expanded system by integrating the Nicla Voice with the Portenta H7 and Arduino Cloud to add functionality and scalability.



Possible expanded glass break detection system

The expanded glass breaking detector uses the Nicla Voice, Portenta H7, and Arduino Cloud to provide practical and scalable safety features for buildings and other enclosed spaces. Its adaptability makes it suitable for various monitoring and security scenarios.

Expanded System Overview

The system begins with the Nicla Voice, which runs a trained machine learning model to detect glass-breaking sounds. Here you will use the model discussed in the [Machine Learning Model Development](#) section, providing a nice foundation for real-world implementations in controlled environments.

Upon recognizing such an event, the Nicla Voice sends a notification via Bluetooth® Low Energy (BLE) to the Portenta H7.



Expanded glass break detection system overview

The Portenta H7 is the host device, bridging the detection hardware with the Arduino Cloud. It receives the BLE notification, processes the alert, and updates the cloud dashboard in real-time.

In addition, it provides an action-based response system that can be customized to comply with the required standards if needed:

Security Alert Call: It is activated upon recognizing the first glass-breaking event. This is to provide an immediate warning that a critical event may have occurred, enabling early intervention.

Lockdown Motor: It is triggered if additional glass-breaking events are detected, escalating the response by activating mechanisms such as reinforced shutters to secure the environment.

This example shows a possible practical example of how the system can be applied to monitor and safeguard buildings or enclosed areas with an escalation mechanism. The approach combines simplicity and adaptability, ensuring the system can address specific needs while providing reliable and responsive features.

Nicla Voice Integration

The Nicla Voice identifies glass-breaking sounds and communicates

When the Nicla Voice recognizes a glass-breaking sound, it triggers the `sendAlert` function to notify the Portenta H7. The following code shows how part of the alert mechanism works:

```
1 void sendAlert(char* label)
2 {
3     Serial.println("Glass break detected");
4
5     // Write to BLE character
6     glassBreakAlert.writeValue(1);
7     NDP.noInterrupts();
8     nicla::leds.begin();
9     nicla::leds.setColor(RED);
10    delay(3000);
11    nicla::leds.end();
12    NDP.interrupts();
13 } else {
14     Serial.println("No real event detected");
15 }
16 }
```

This part of the code ensures that the Nicla Voice communicates detected events to the host device, allowing you to have rapid responses. In this case, the function ensures the Nicla Voice can notify the Portenta H7 as soon as a glass-breaking sound is detected.

The full implementation of the Nicla Voice includes the BLE setup, NDP configuration, and event handling routines. The complete code for the Nicla Voice for Arduino IDE is as follows:

```

1  /**
2   Glass-Breaking Detection
3
4   Nicla Voice code to interface with the ML Node
5   and notifying the Portenta H7
6  */
7
8  #include "NDP.h"
9  #include <ArduinoBLE.h>
10
11 // BLE Service for Glass-Breaking Detection
12 BLEService alertService("Glass-Breaking Detection")
13
14 // BLE Alert Characteristic
15 BLEByteCharacteristic alertCharacteristic("Glass-Breaking Detection")
16
17 // Global Parameters
18 const bool lowestPowerMode = false;
19
20 /*****
21  * Nicla Voice ML Node
22  *****/
23
24 /**
25  Inference Interrupt
26  It triggers the built-in interrupt
27  - Trigger label: NN0_INTERRUPT
28  */

```

If you have not yet deployed and tested the Nicla Voice with trained machine learning model for glass-break detection, please refer to how to [deploy the model](#) or to its [development process](#).

Portenta H7 Integration

The Portenta H7 is responsible for processing BLE notifications from the Nicla Voice and interfacing with the Arduino Cloud. It interprets alerts and manages the system's response through cloud updates and hardware control routines with event level details.

The following code snippet shows

specific alerts:

```
1  if (AlertValue == 1) {  
2    alertStatus = true;  
3    if (!securityAlert) {  
4      securityAlert = true;  
5      GlassEvent = "Security Alert";  
6  
7      // Insert Code with p  
8  
9      Serial.println("Security Alert");  
10   } else {  
11     lockdownMotor = true;  
12     GlassEvent = "Lockdown";  
13  
14     // Insert Code with p  
15  
16     Serial.println("Lockdown");  
17   }  
18 }
```

Initial Alert: When the first glass-breaking event is detected, the system sets `SecurityAlert` to `true`. This activates the **Security Alert Call** routine, warning connected users or systems via the cloud dashboard.

Escalated Alert: If further glass-breaking events occur, the system sets `LockdownMotor` to `true`, activating physical mechanisms like shutters to secure the environment.

The logic exemplifies how the subsequent events escalated appropriately, providing a dynamic and layered response to potential threats and handle possible complex scenarios.

The full implementation for the

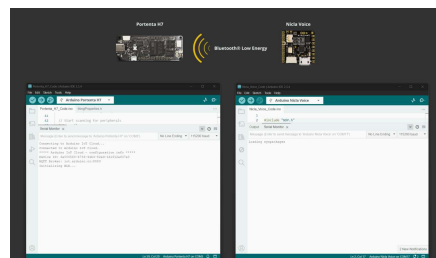
response routines. The complete code for the Portenta H7 is as follows:

```

1  /**
2      Glass-Breaking Detect
3
4      This code sets the Po
5          from the Nicla Voic
6      Arduino Cloud. The
7          activating a Securi
8  */
9
10 #include <ArduinoIoTCloud>
11 #include <Arduino_Conne
12 #include "thingPropert
13 #include <ArduinoBLE.h>
14
15 // Global Parameters
16 byte AlertValue = 0;
17
18 /**
19     Main setup section
20 */
21 void setup() {
22     // Initialize serial
23     Serial.begin(115200);
24     while (!Serial);
25
26     // Cloud connection s
27     pinMode(LEDG, OUTPUT)
28

```

Once the Nicla Voice and Portenta H7 are ready with the scripts and configurations, you can expect to observe behavior similar to the animation below:



The expanded system integrates cloud connectivity via the Portenta

Arduino Cloud Integration

In this example, you can view real-time event logs, track BLE connection status, and observe the *Security Alert* or *Lockdown* activation:

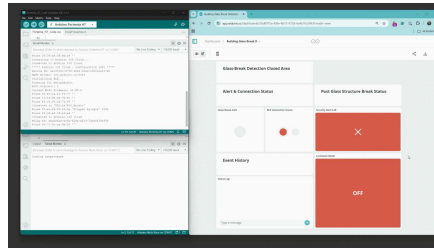
The dashboard logs and displays events as they occur, allowing you to stay informed about glass-breaking incidents.

Security Alert and Lockdown Mechanism Indicators:

You can monitor the activation status of the *Security Alert* and *Lockdown* mechanism directly from the dashboard.

The cloud integration allows you to receive updates and take action from anywhere, making the system more

the inference process and communication of Nicla Voice and the Portenta H7:



Full Glass-Breaking Detector Resources

All the codes and files of this application note can be found below:

[Base glass-breaking detector example](#)

[Extended glass-breaking detector with Arduino Cloud](#)

The Machine Learning Tools project is public [here](#) by Aurelien Lequertier. You can clone it and modify it to adapt it to your needs by improving the dataset or model architecture for a custom deployment.

Conclusion

This application note explains how to use Edge Impulse and Nicla Voice for real-time sound classification directly on the device. The process is straightforward, covering from managing data to running the model, making it accessible even for those with limited experience in machine learning.

While this application note focuses on detecting glass-breaking sounds, the same approach can be used for

With the Nicla Voice and Edge Impulse, you can develop flexible systems for various safety and monitoring needs.

Next Steps

This system can be adapted to recognize other sounds or events by retraining the model with a new dataset. Potential applications include:

- Detecting specific machinery noises in industrial settings.

- Monitoring animal sounds in wildlife conservation.

- Recognizing spoken keywords for voice-controlled systems.

You can also explore the [Door Intruder Detector Using ML with the Nicla Voice](#) application note that shows how machine learning can detect door-opening and intrusion events, providing additional insights into creating versatile sound detection systems for enhanced domestic security.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

